

Java Inheritance (Abstract Classes & Interfaces) Extra Practice Practical Exercises

Instructions

Complete the following exercises to reinforce your understanding of abstract classes and interfaces in Java. Implement your solutions in a Java development environment.

Exercise 1: Implementing an Abstract Class

1. Create an abstract class `Payment` with the following:
 - An instance variable `amount`.
 - A constructor to initialize `amount`.
 - An abstract method `processPayment()`.
2. Create two subclasses:
 - `CreditCardPayment`: Implements `processPayment()` to display a credit card transaction message.
 - `PayPalPayment`: Implements `processPayment()` to display a PayPal transaction message.
3. In the `main` method, create objects of both subclasses and call their methods.

Exercise 2: Implementing an Interface

1. Define an interface `Refundable` with a method `refund()`.
2. Modify `CreditCardPayment` so that it implements `Refundable`, allowing a refund operation.
3. In the `main` method, check if an object of `CreditCardPayment` is refundable using `instanceof`, then call `refund()`.

Exercise 3: Combining Abstract Classes and Interfaces

1. Modify the `Payment` class to include:
 - A concrete method `displayAmount()` that prints the payment amount.
2. Create a new interface `DigitalWallet` with a method `walletInfo()`.
3. Modify `PayPalPayment` so that it implements `DigitalWallet`:
 - The `walletInfo()` method should display the user's email.
4. In the `main` method, create an object of `PayPalPayment` and call all available methods.

Exercise 4: Multiple Inheritance using Interfaces

1. Define an interface `CurrencyConverter` with a method `convertCurrency()`.
2. Modify `PayPalPayment` so that it implements `CurrencyConverter` and provides an exchange rate conversion.
3. In the `main` method, call `convertCurrency()` for a PayPal transaction.

Exercise 5: Payment Processing System

Design a payment processing system that includes different types of transactions with various payment methods:

1. Modify `Payment` to include:
 - A method `generateReceipt()` that prints a transaction receipt.
2. Define an interface `MobilePayment` with a method `mobileTransaction()`.
3. Create a new subclass `GooglePayPayment` that extends `Payment` and implements `MobilePayment`.
4. In the `main` method:
 - Create objects of `CreditCardPayment`, `PayPalPayment`, and `GooglePayPayment`.
 - Call their respective methods.